

Subiecte tehnice: gălbenuș Proxy distribuit, modelul Publish-Subscribe din comunicația asimetrică, inversion of control (IoC), tehnică ORM pt. maparea membrilor stării, tehnică ORM pt. maparea măștinilor, structura unui token JWT, Dependency Injection, ORM Impedance Mismatch, modelul Point-to-point din com. asimetrică, broker de mesaje, constr. stateless pt. REST

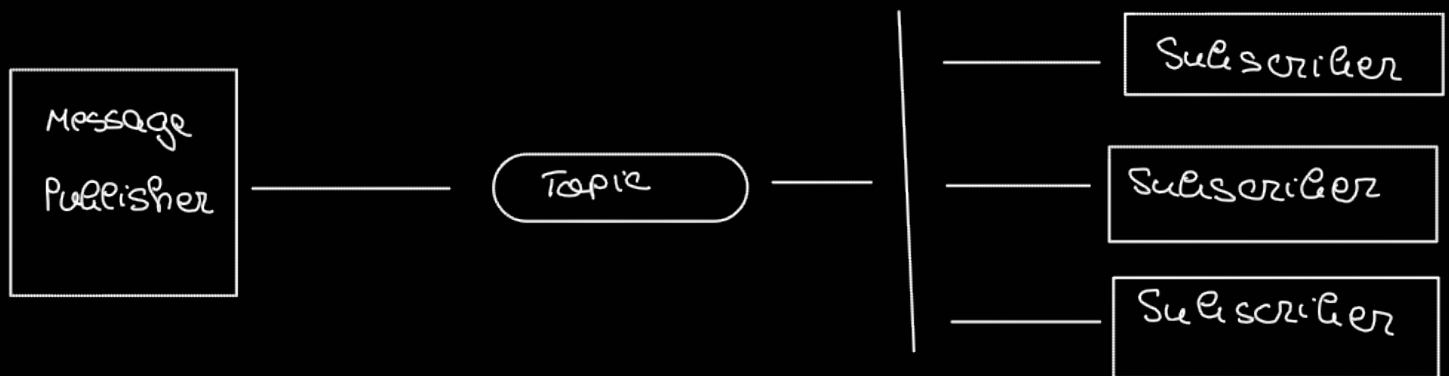
1. Gălbenuș Proxy distribuit:

- gălbenuș proxy are rolul de a oferi, pt. un obiect existent, un surrogat sau un înlocuitor, în scopul controlării accesului la acesta.
- în cazul gălbenușului Remote Proxy, acesta oferă un înlocuitor local pt. un obiect care funcționează într-un alt spațiu de adresă sau de memorie.
- caract. și module de funcționare:
 - transparență pt. client
 - clientul care face cererea crede că interacționează direct cu obiectul real, dar în realitate face acest lucru cu proxy-ul.
 - clientul poate apela metodele pe acest proxy local exact ca și cum ar efectua operațiunile locale.
 - responsabilitatea de codificare și transmitere
 - proxy-ul preia cererea clientului și este responsabil de codificarea acesteia, pe care apoi o trimite prin rețea către obiectul aflat la distanță.
 - returnarea rezultatului
 - după ce proxy-ul transformă cererea locală într-o cerere la distanță și o trimite, acesta așteaptă procesarea pe server, obține rez. cererii și îl transmite clientului.

2. Publish - Subscribe Model

Overview: mesajele trimise asimetric au o sursă și o destinație (cărți și topic-uri); pentru fiecare tip de destinație există un model specific de trimitere a mesajelor → pentru topic-uri avem modelul **publish - subscribe**

În modelul **publish - subscribe**, mesajele sunt trimise unui topic. Mai mulți destinatori pot aștepta și vor primi un mesaj de la un topic, asemenea cărților, dar toți acești destinatori vor primi o copie a aceluiași mesaj. Expeditorul nu știe care sunt destinatorii sau cum vor fi procesate mesajele, el doar ca mesajul se va trimite unui topic.



3. Point - To - Point Model

În modelul **point - to - point**, fiecare mesaj are exact un expeditor și un destinator. Când brokerul de mesaje primește un mesaj, acesta pune mesajul într-o coadă. Când destinatorul cere următorul mesaj, el este scos din coadă și trimis destinatorului. Trecând de la faptul că e ieșirea din coadă mesajul este gata, nu se poate garanta livrarea sau decât către un singur destinator.

Pot exista mai mulți destinatori care așteaptă la coadă, însă fiecare din aceștia va avea un mesaj diferit de procesat.

Dacă mai mulți destinatori așteaptă procesarea unui mesaj din codă, nu se știe exact ce mesaj va fi procesat fiecare. Acest lucru permite aplicărilor să mărească capacitatea de procesare a mesajelor prin adăugarea unui nou destinator.



4. Inversarea Control (IaC)

Cunoscut și sub numele de **dependency injection**, IaC este procesul prin care obiectele își definesc asocierile fie prin param. constructorilor, fie prin arg. unei metode de tip factory sau prin prop. de tip set, care trebuie apelate imediat după crearea obiectului.

Un container "injectează" aceste dependențe când creează obiectul, procesul fiind invers celui tradițional în care obiectul este responsabil de instanțierea sau localizarea acestora.

Un exemplu în acest sens sunt bean-urile Spring care reprez. doar un obiect Java obișnuit care este instanțiat, amenajat și gestionat de containerul Spring IaC.

5ORM Impedance Mismatch

ORM Impedance Mismatch se referă la diferența dintre următoarele abordări: paradigma orientată obiect (promovează dez. aplicațiilor folosind obiecte care păstrează date, dar combină și logica aplicației) și bazele de date relaționale

↳ stocarea datelor în tabele și manipularea datelor folosind proceduri stocate și interogări SQL

Un exemplu în acest sens ar fi faptul că în paradigma orientată obiect obiectele sunt traversate folosind relațiile dintre ele, iar în paradigma relațională se folosește operația de join.

6. Tehnica ORM de mapare a măștenirii

Există mai multe tehnici de mapare a măștenirii, precum: într-o singură tabelă, fiecare clasă comută cu tabela ei, fiecare clasă într-o tabelă sau maparea într-o structură de tabele generică.

Se va aborda tehnica de mapare a fiecărei clase într-o tabelă. Astfel, atributele din clasă se mapază direct la coloanele corespunzătoare din tabelă.

Pentru a lega înregistrările în tabele, fiecare tabelă va avea cheia sa primară, însă vor fi legate printr-o cheie străină (în tabelă derivată) care păstrează „asocierea” cu tabelul corespunzător clasei de bază.

Pentru regăsirea informațiilor din mai multe tabele se poate utiliza operația de join.

AVANTAJE:

- toate căpădamele sunt relevante pt. fiecare înregistrare, tablele sunt mai ușor de înțeles și nu se rălăsește spațiul în mod ineficient
- relația dintre modelul orientat obiect și baza relațională este ușor de înțeles

DEZAVANTAJE (o parte)

- e necesară utilizarea mai multor table pt. încărcarea unui obiect din mediu persistent
- orice restructurare (mutarea compunilor în ierarhia de clase) cauzează modificarea structurii bazei de date

7. Tehnica ORM de mapare la prop. statice

O metodă de mapare ar fi utilizarea unei table cu o singură înregistrare pentru fiecare clasă.

Avantajul este că e o tehnică rapidă, ce permite acces rapid, însă creează multe table mici.

A
<u>static Attr 1</u> name

Static Table A

static Attr 1
1

B
<u>static Attr 1</u> <u>static Attr 2</u>

Static Table B

static Attr 1	static Attr 2
23	Value A

8. Protocol de mesaje

În cazul trimiterii asimetrice de mesaje există 2 actori principali: Emisora de mesaje și destinatarul. Atunci când o aplicație trimite un mesaj, se trimite unui Emisora de mesaje.

Protocolul de mesaje asigură livrarea mesajului la destinația specificată, permițând expeditorului continuarea execuției.

9. Structura unui token JWT

JSON Web Token este un standard care definește un mod compact și independent de a transmite informații între participanți ca un obiect JSON.

Este format din 3 părți separate printr-un punct: Header, payload și semnătura digitală.

Header - ue conține informații despre tipul de token și alg. de criptare folosită pentru conținut.

Ex :
{
 "alg": "SHA256",
 "typ": "JWT"
}

Payload - ue conține informații despre entitate ce poate fi verificată. (aka claims)

Ex :
{
 "name": "John",
 "admin": true
}

Semnătura digitală este folosită pentru a verifica expeditorul și pt. a verifica dacă mesajul a fost modificat pe parcurs. Se codifică Base64 header-ul și payload-ul și se criptează folosind alg. specificat în header.

10. Constrângerea stateless pentru REST

Acest lucru înseamnă că fiecare cerere va fi tratată ca și cum ar fi o cerere nouă. Fără sesiune, fără istoric. Fiecare cerere trimisă de la client la server trebuie să conțină toate informațiile necesare pt. a înțelege cererea și nu poate folosi inf. legate de context păstrate pe server. Starea sesiunii va fi păstrată în întregime la client.

11. Protocolul websocket

Protocolul WebSocket funcționează oferind o conexiune persistentă, bi-direcțională și full-duplex de la un client la un server. Utilizând acest protocol, programarea web devine bazată pe eveniment, având caracter stateful.

1. Stabilirea conexiunii prin Handshake

- clientul trimite o cerere HTTP pt. un handshake websocket
- dacă serverul acceptă cererea de actualizare a protocolului ⇒ răspuns HTTP 101 Switching Protocols → din acest moment, amândoi participanți pot trimite / primi mesaje în orice moment de timp

2. Se definește o formată URI nouă.

- Pentru 80 este folosit pt. conexiuni nesecurizate,

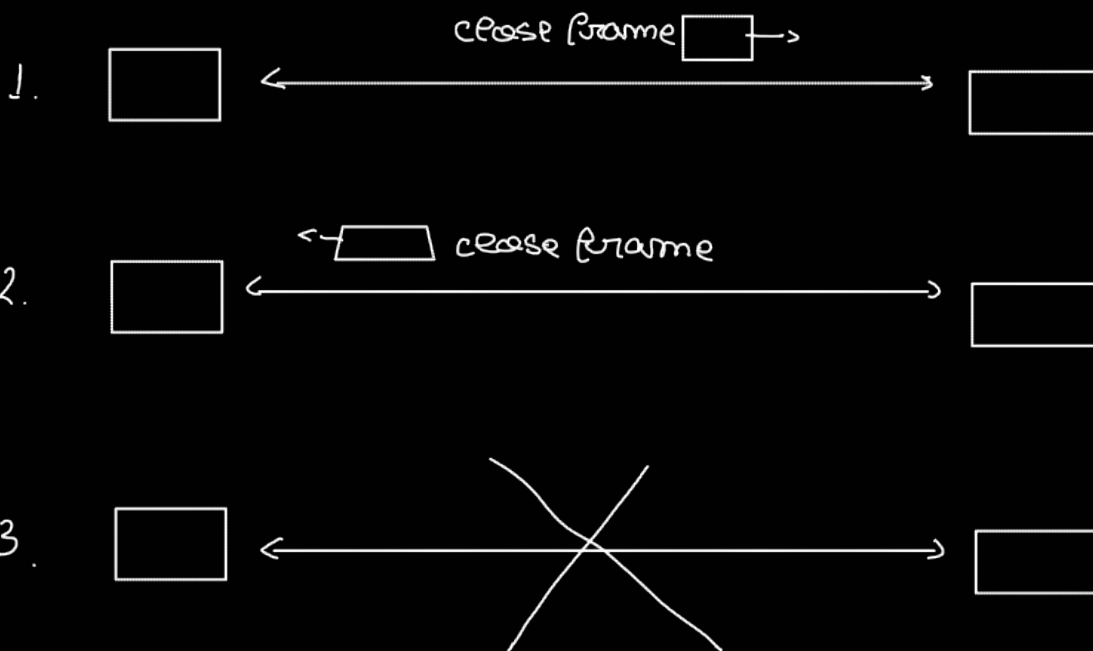
→ (ws://)

3. Mesajele WebSocket

- un mesaj e compus dintr-o secu. de unul sau mai multe fragmente de mesaj / date numite "frames"; fiecare frame conține info. cum ar fi:
 - dimensiunea frame-ului
 - primul frame din mesaj conține tipul mesajului (text sau binar)
 - un flag (FIN) ce indică dacă frame-ul curent este sau nu ultimul din mesaj

4. Închiderea unui WebSocket

- poate să fie inițiator de închidere participant la conexiune (clientul sau serverul)
- este trimis un frame special (frame close) către celălalt participant



12. Șablonul Data Transfer Object (DTO)

Un DTO este un obiect care conține informația ce trebuie transmisă între unul sau mai multe procese pentru a reduce nr. de apeluri. Acesta conține, în general, multe atribute și metode de tip get/set.

când obiectul normate are nevoie de date, cere DTO-ul corespunzător. DTO poate să conțină mai multă inf. decât este necesară la acel apel, dar tre. să conțină totă inf. de care va avea nevoie obiectul normate o perioadă.

un DTO conține de obicei inf. provenind de la mai multe obiecte din model

